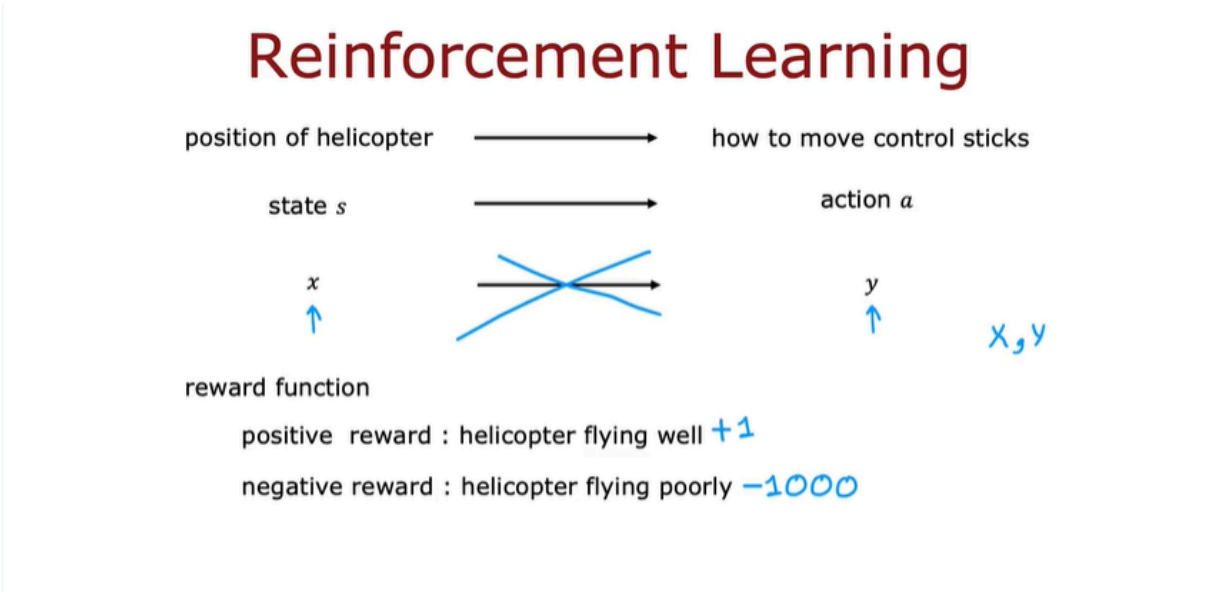
Reinforcement learning introduction

What is Reinforcement Learning?

Let's start with an example. Here's a picture of an autonomous helicopter. This is actually the Stanford autonomous helicopter, weighs 32 pounds and it's actually sitting in my office right now. Like many other autonomous helicopters, it's instrumented with an onboard computer, GPS, accelerometers, and gyroscopes and the magnetic compass so it knows where it is at all times quite accurately.

And if I were to give you the keys to this helicopter and ask you to write a program to fly it, how would you do so? Radio controlled helicopters are controlled with joysticks like these and so the task is ten times per second you're given the position and orientation and speed and so on of the helicopter. And you have to decide how to move these two control sticks in order to keep the helicopter balanced in the air. By the way, I've flown radio controlled helicopters as well as quad rotor drones myself. And radio controlled helicopters are actually quite a bit harder to fly, quite a bit harder to keep balanced in the air.



So how do you get a helicopter to fly itself using reinforcement learning? The task is given the position of the helicopter to decide how to move the control sticks.

In reinforcement learning, we call the position and orientation and speed and so on of the helicopter the state s.

And so the task is to find a function that maps from the state of the helicopter to an action a, meaning how far to push the two control sticks in order to keep the helicopter balanced in the air and flying and without crashing.

- One way you could attempt this problem is to use supervised learning. It turns out this is not a great approach for autonomous helicopter flying. But you could say, well if we could get a bunch of observations of states and maybe have an expert human pilot tell us what's the best action y to take. You could then train a neural network using supervised learning to directly learn the mapping from the states s which I'm calling x here, to an action a which I'm calling the label y here.
- But it turns out that when the helicopter is moving through the air is actually very ambiguous, what is the exact one right action to take. Do you tilt a bit to the left or a lot more to the left or increase the helicopter stress a little bit or a lot? It's actually very difficult to get a data set of x and the ideal action y. So that's why for a lot of task of controlling a robot like a helicopter and other robots, the supervised learning approach doesn't work well and we instead use reinforcement learning.

Now a key input to a reinforcement learning is something called the reward or the reward function which tells the helicopter when it's doing well and when it's doing poorly. So the way I like to think of the reward function is a bit like training a dog. When I was growing up, my family had a dog and it was my job to train the dog or the puppy to behave. So how do you get a puppy to behave well?

Well, you can't demonstrate that much to the puppy. Instead you let it do its thing and whenever it does something good, you go, good dog. And whenever they did something bad, you go, bad dog. And then hopefully it learns by itself how to do more of the good dog and fewer of the bad dog things. So training with the reinforcement learning algorithm is like that. When the helicopter's flying well, you go, good helicopter and if it does something bad like crash, you go, bad helicopter. And then it's the reinforcement learning algorithm's job to figure out how to get more of the good helicopter and fewer of the bad helicopter outcomes.

One way to think of why reinforcement learning is so powerful is you have to tell it what to do rather than how to do it. And specifying the reward function rather than the optimal action gives you a lot more flexibility in how you design the system. Concretely for flying the helicopter, whenever it is flying well, you may give it a reward of plus one every second it is flying well. And maybe whenever it's flying poorly you may give it a negative reward or if it ever crashes, you may give it a very large negative reward like negative 1,000. And so this would incentivize the helicopter to spend a lot more time flying well and hopefully to never crash.

Applications

Applications

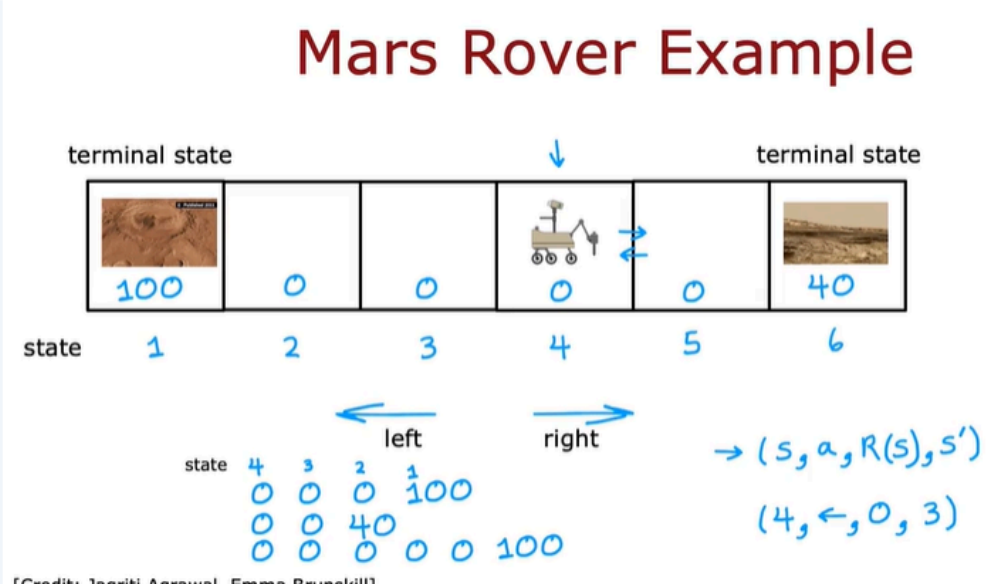
- Controlling robots
- Factory optimization
- Financial (stock) trading
- Playing games (including video games)

So that's reinforcement learning. Even though reinforcement learning is not used nearly as much as supervised learning, it is still used in a few applications today. And the key idea is rather than you needing to tell the algorithm what is the right output y for every single input, all you have to do instead is specify a reward function that tells it when it's doing well and when it's doing poorly.

Mars rover example

(S, a, R(S), S′)

- S: Current state
- a: action
- R(S): reward
- S′: new state



We'll develop reinforcement learning using a simplified example inspired by the Mars rover. In this application, the rover can be in any of six positions, as shown by the six boxes here. The rover, it might start off, say, in disposition into fourth box shown here

. The position of the Mars rover is called the state in reinforcement learning, and I'm going to call these six states, state 1, state 2, state 3, state 4, state 5, and state 6, and so the rover is starting off in state 4. Now the rover was sent to Mars to try to carry out different science missions. It can go to different places to use its sensors such as a drill, or a radar, or a spectrometer to analyze the rock at different places on the planet, or go to different places to take interesting pictures for scientists on earth to look at.

In this example, state 1 here on the left has a very interesting surface that scientists would love for the rover to sample.

State 6 also has a pretty interesting surface that scientists would quite like the rover to sample, but not as interesting as state 1. We would more likely to carry out the science mission at state 1 than at state 6, but state 1 is further away. The way we will reflect state 1 being potentially more valuable is through the reward function.

The reward at state 1 is a 100, and the reward at stage 6 is 40, and the rewards at all of the other states in-between, I'm going to write as a reward of zero because there's not as much interesting science to be done at these states 2, 3, 4, and 5.

On each step, the rover gets to choose one of two actions. It can either go to the left or it can go to the right. The question is, what should the rover do? In reinforcement learning, we pay a lot of attention to the rewards because that's how we know if the robot is doing well or poorly. Let's look at some examples of what might happen if the robot was to go left, starting from state 4. Then initially starting from state 4, it will receive a reward of zero, and after going left, it gets to state 3, where it receives again a reward of zero.

Then it gets to state 2, receives the reward is 0, and finally just to state 1, where it receives a reward of 100. For this application, I'm going to assume that when it gets either state 1 or state 6, that the day ends. In reinforcement learning, we sometimes call this a terminal state, and what that means is that, after it gets to one of these terminals states, gets a reward at that state, but then nothing happens after that. Maybe the robots run out of fuel or ran out of time for the day, which is why it only gets to either enjoy the 100 or the 40 reward, but then that's it for the day. It doesn't get to earn additional rewards after that.

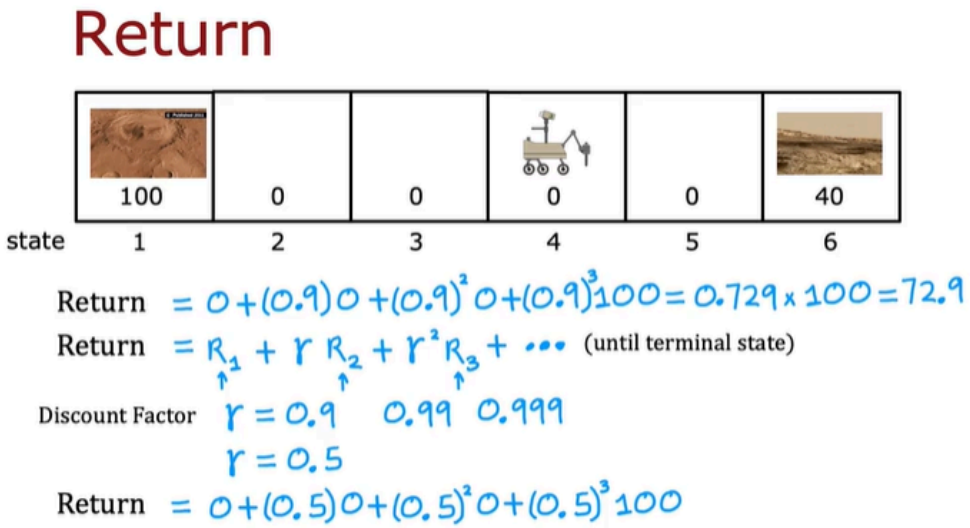
Now instead of going left, the robot could also choose to go to the right, in which case from state 4, it would first have a reward of zero, and then it'll move right and get to state 5, have another reward of zero, and then it will get to this other terminal state on the right, state 6 and get a reward of 40. But going left and going right are the only options.

One thing the robot could do is it can start from state 4 and decide to move to the right. It goes from state 4-5, gets a reward of zero in state 4 and a reward of zero in state 5, and then maybe it changes its mind and decides to start going to the left as follows, in which case, it will get a reward of zero at state 4, at state 3, at state 2, and then the reward of 100 when it gets to state 1. In this sequence of actions and states, the robot is wasting a bit of time. So this maybe isn't such a great way to take actions, but it is one choice that the algorithm could pick, but hopefully you won't pick this one.

To summarize, at every time step, the robot is in some state, which I'll call S, and it gets to choose an action, and it also enjoys some rewards, R of S that it gets from that state. As a result of this action, it to some new state S prime. As a concrete example, when the robot was in state 4 and it took the action, go left, maybe didn't enjoy the reward of zero associated with that state 4 and it won't have any new state 3. When you learn about specific reinforcement learning algorithms, you see that these four things, the state, action, the reward and next state, which is what happens basically every time you take an action that just be a core elements of what reinforcement learning algorithms will look

at when deciding how to take actions. Just for clarity, the reward here, R of S, this is the reward associated with this state. This reward of zero is associated with state 4 rather than with state 3. That's the formalism of how a reinforcement learning application works.

The Return in reinforcement learning



🔁 Return in Reinforcement Learning

Definition:

The **return** `Gt` at time step `t` is the **total accumulated reward** the agent expects to receive from time `t` onward. It's used to assess how good a particular state or action is in terms of long-term reward.

📐 Mathematical Formula

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

Or, in summation form:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

🧩 Components

- G_t: Reward at time step `t`
- γ (gamma): **Discount factor**, where `0 ≤ γ ≤ 1`
  - If `γ = 0`: Only immediate rewards are considered
  - If `γ ≈ 1`: Future rewards are considered nearly as important as immediate ones

---

🎯 Purpose

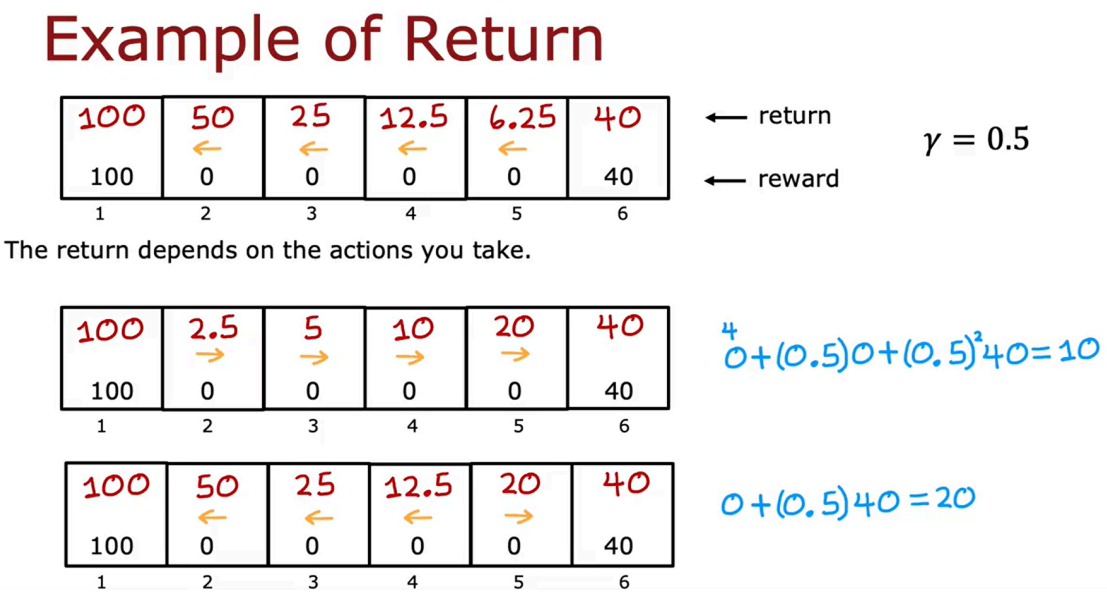
The return `Gt` is used to:

- Evaluate how **valuable** a state or action is
- Help the agent **maximize long-term rewards**
- Form the basis of **value functions**:
  - **State-value function**:

$$V(s) = \mathbb{E}[G_t \mid S_t = s]$$

  - **Action-value function**:

$$Q(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$$



To summarize, the return in reinforcement learning is the sum of the rewards that the system gets, weighted by the discount factor, where rewards in the far future are weighted by the discount factor raised to a higher power.

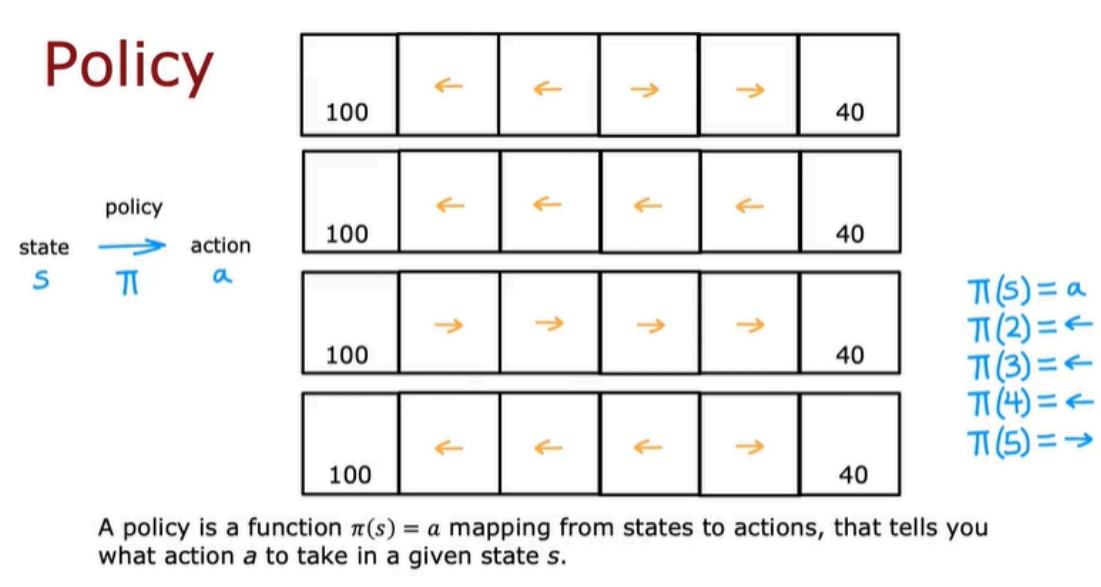
Now, this actually has an interesting effect when you have systems with negative rewards. In the example we went through, all the rewards were zero or positive. But if there are any rewards are negative, then the discount factor actually incentivizes the system to push out the negative rewards as far into the future as possible.

Taking a financial example, if you had to pay someone $10, maybe that's a negative reward of minus 10. But if you could postpone payment by a few years, then you're actually better off because $10 a few years from now, because of the interest rate is actually worth less than $10 that you had to pay today.

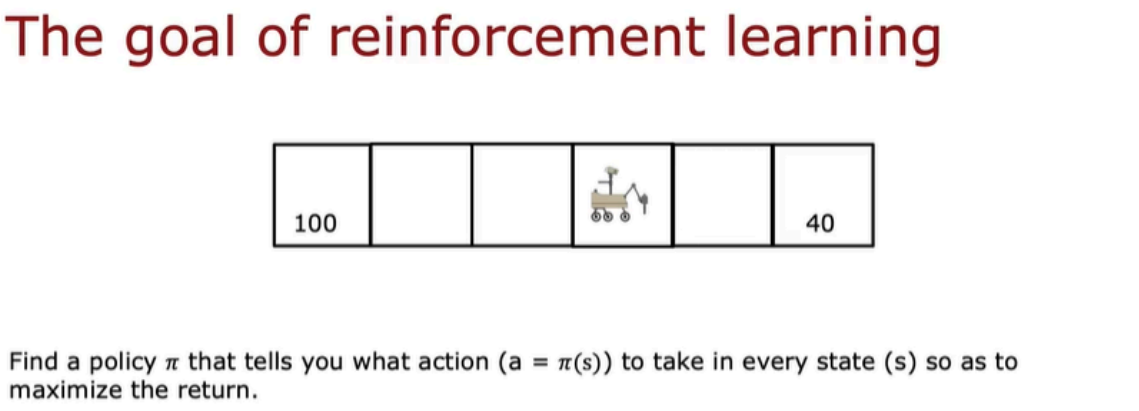
For systems with negative rewards, it causes the algorithm to try to push out the make the rewards as far into the future as possible. For financial applications and for other applications, that actually turns out to be right thing for the system to do.

Making decisions: Policies in reinforcement learning

A policy

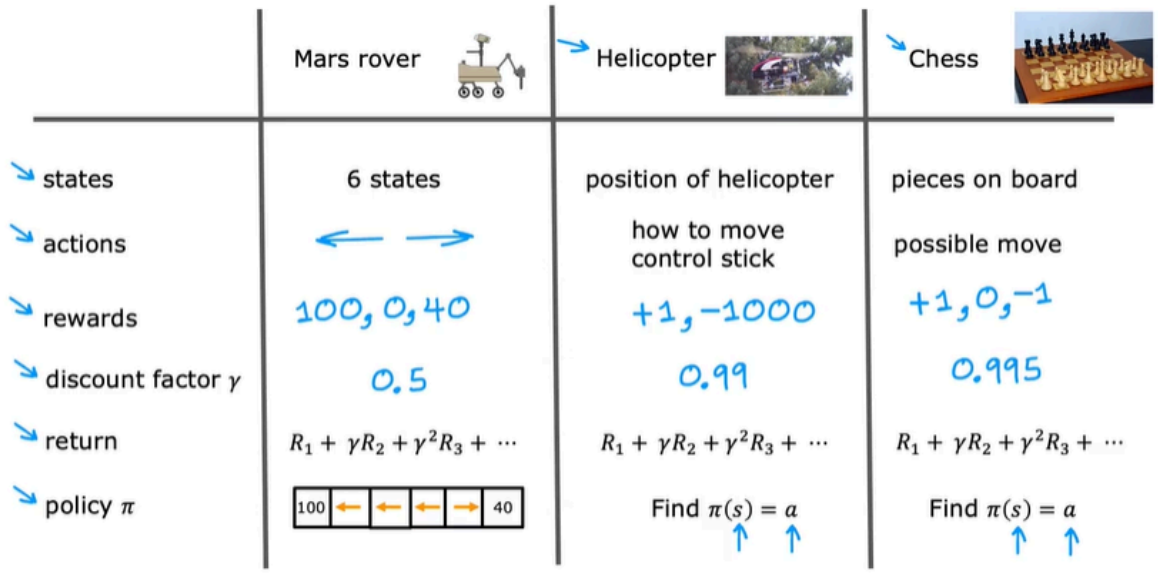


A policy is a function $\pi(s) = a$ mapping from states to actions, that tells you what action a to take in a given state s.



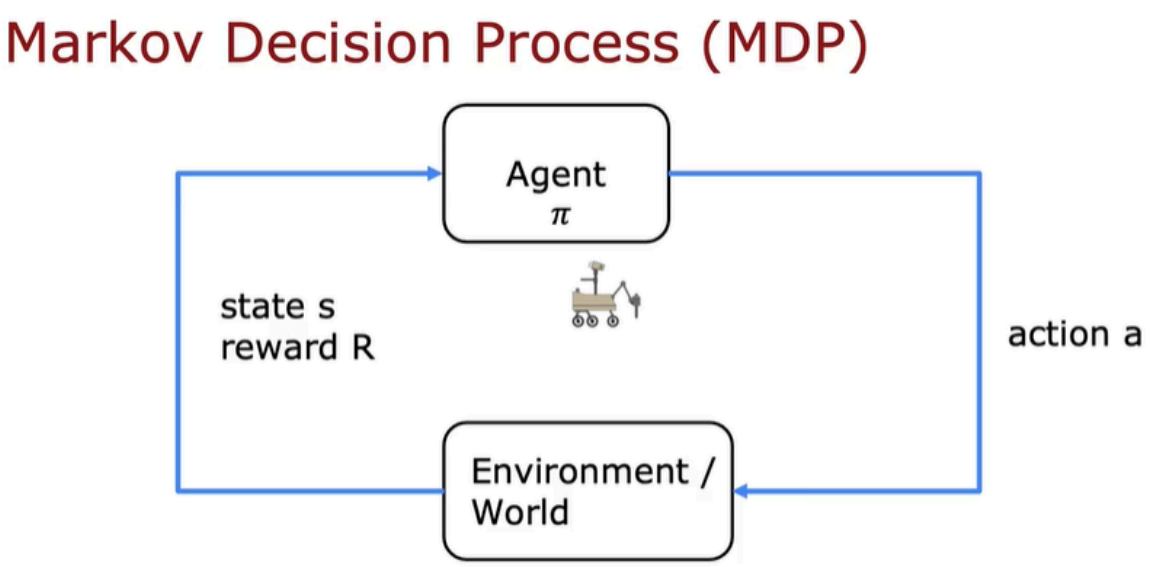
By the way, I don't know if policy is the most descriptive term of what π is, but it's one of those terms that's become standard in reinforcement learning. Maybe calling Pi a controller rather than a policy would be more natural terminology but policy is what everyone in reinforcement learning now calls this.

Review of key concepts



Once again, the goal is given a board position to pick a good action using a policy Pi. This formalism of a reinforcement learning application actually has a name.

Markov Decision Process (MDP)



The term Markov in the MDP or Markov decision process refers to that the future only depends on the current state and not on anything that might have occurred prior to getting to the current state. In other words, in a Markov decision process, the future depends only on where you are now, not on how you got here.

One other way to think of the Markov decision process formalism is that we have a robot or some other agent that we wish to control and what we get to do is choose actions a and based on those actions, something will happen in the world or in the environment, such as our position in the world changes or we get to sample a piece of rock and execute the science mission.

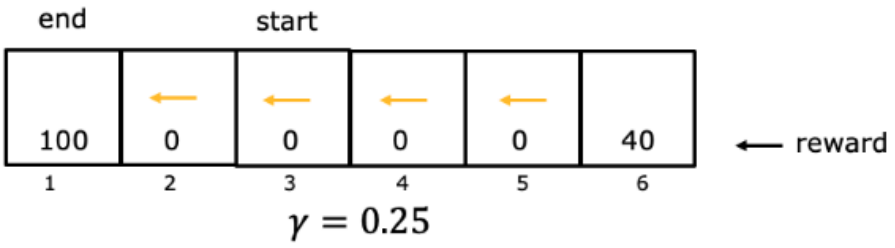
The way we choose the action a is with a policy Pi and based on what happens in the world, we then get to see or we observe back what state we're in, as well as what rewards are that we get.

You sometimes see different authors use a diagram like this to represent the Markov decision process or the MDP formalism but this is just another way of illustrating the set of concepts that you learn about in the last few videos.

You now know how a reinforcement learning problem works. In the next video we'll start to develop an algorithm for picking good actions. The first step toward that will be to define and then eventually learn to compute the state action value function.

This turns out to be one of the key quantities for when we want to develop a learning algorithm. Let's go onto the next video to see what is this, state action value function.

4. Given the rewards and actions below, compute the return from state 3 with a discount factor of $\gamma = 0.25$.



- ○ 0
- ◉ 6.25

If starting from state 3, the rewards are in states 3, 2, and 1. The return is $0 + (0.25) \times 0 + (0.25)^2 \times 100 = 6.25$.